

Tecnicatura Universitaria en Programación a Distancia

Universidad Tecnológica Nacional (UTN)

TRABAJO PRÁCTICO INTEGRADOR

Programación 1

Título del Proyecto: Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Integrantes: Alexis Leonel Ortiz | Lautaro Nicolas Palenzuela

Fecha de Entrega: 15/7

Links obligatorios:

- Repositorio GitHub: [Enlace]
- Video Demostrativo (10-15 min): [Enlace]

ÍNDICE

1. Carátula e Información de Proyecto
2. Marco Teórico Ampliado
3. Decisiones Técnicas y Arquitectura del Sistema
4. Dificultades Técnicas Encontradas y Resoluciones
5. Conclusiones Grupales y Aprendizajes
6. Bibliografía y Webgrafía

1. MARCO TEÓRICO

El desarrollo de software estructurado requiere fundamentar el uso de herramientas lógicas en base a la eficiencia de memoria y claridad algorítmica. A continuación, se detallan los pilares conceptuales implementados en esta aplicación:

1.1 Estructuras de Datos Lineales y Dinámicas (Listas)

Una lista en Python representa una estructura secuencial, indexada y mutable de elementos. En el ecosistema del proyecto, actúa como la base de datos central en memoria RAM, permitiendo mutabilidad dinámica mediante la inserción de registros en tiempo de ejecución a

través del método secuencial nativo `.append()` sin necesidad de dimensionamiento previo.

1.2 Estructuras de Datos Compuestas (Diccionarios)

Un diccionario es una colección no ordenada de elementos estructurados bajo el paradigma clave-valor (hash map). Esta herramienta otorga un acceso semántico directo a los atributos del dominio. La combinación de ambas estructuras conforma una **Lista de Diccionarios**, abstrayendo una matriz bidimensional (tabla relacional) de manera nativa:

Estructura Colectora	Elemento de Registro	Atributos de Datos (Claves)
Lista (Mutable / Ordenada)	Diccionario (Clave-Valor)	nombre (str), poblacion (int), superficie (int), continente (str)

1.3 Modularización y Funciones de Responsabilidad Única

La modularización segmenta el flujo de control general del programa en subprocesos con un alcance aislado y acotado. Bajo el principio de diseño de que una función debe responder a una única responsabilidad conceptual, se logra un código desacoplado. Esto facilita el aislamiento de fallos (debugging), optimiza las pruebas unitarias y elimina la redundancia de código en las interfaces de usuario.

1.4 Persistencia en Archivos CSV

La persistencia se logra desviando el flujo de datos desde la memoria volátil (RAM) hacia el almacenamiento secundario persistente a través del formato CSV (Comma-Separated Values). El tratamiento nativo de archivos de texto plano mediante buffers limpios con sentencias `with open()` asegura el cierre automático de descriptores de archivos del sistema operativo, previniendo fugas de memoria o bloqueos lógicos de archivos.

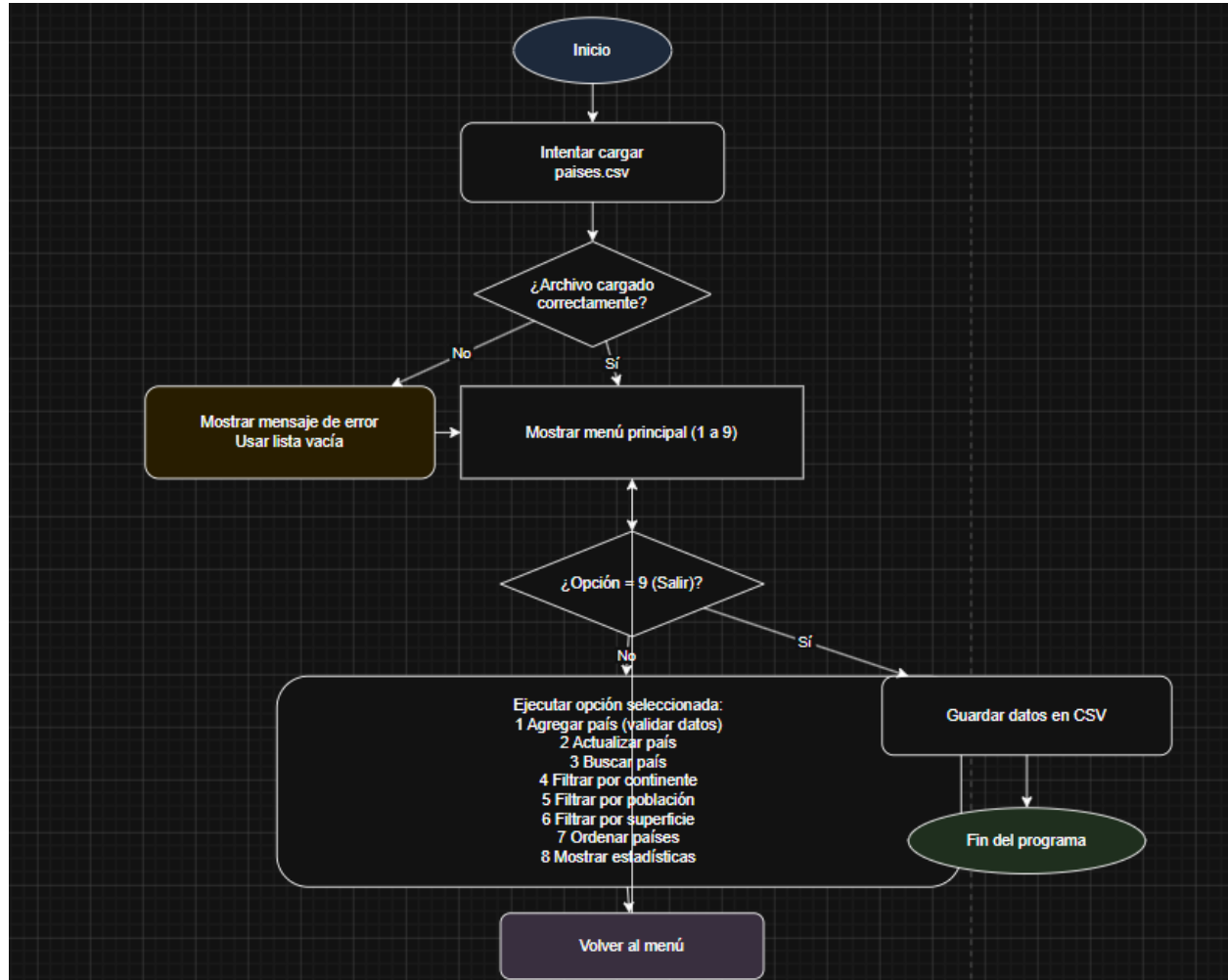
1.5 Validación de Entradas y Manejo Operacional de Excepciones

La robustez de una aplicación se mide en función de su tolerancia a fallos ante entradas inesperadas del operador humano. El uso de bloques de control de excepciones condicionales (estructuras `try-except` para atrapar errores de tipo `ValueError` o `FileNotFoundError`) previene el colapso abrupto del hilo de ejecución del intérprete de Python, transformando fallas críticas en mensajes informativos legibles en consola.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

El sistema se diseñó siguiendo una arquitectura monolítica modularizada dividida por responsabilidades lógicas bien marcadas. El flujo secuencial de operaciones se rige bajo el siguiente esquema lógico:

2. 1 Diagrama del Flujo de Operaciones Principales



Inicio del programa: se ejecuta la aplicación y comienza el flujo principal.

Carga inicial de datos: la función `cargar_desde_csv()` intenta cargar el archivo `países.csv`. Si el archivo no existe o ocurre un error, se muestra un mensaje y se inicializa una lista vacía.

Ingreso al menú principal: el sistema entra en un bucle `while True`, mostrando al usuario las opciones disponibles del menú (1 a 9).

Opción 1 – Agregar país: se solicitan los datos del país, se validan los campos numéricos y se controla que no existan duplicados antes de incorporarlo a la lista.

Opción 2 – Actualizar país: se busca el país por nombre y, si existe, se modifican los valores almacenados.

Opción 3 – Buscar país: se realiza una búsqueda flexible por nombre utilizando coincidencias parciales.

Opción 4 – Filtrar países: se filtra la lista según continente, rango de población o superficie.

Opción 5 – Ordenar países: se ordenan los países por nombre, población o superficie, en forma ascendente o descendente.

Opción 6 – Estadísticas: se calculan valores máximos, mínimos, promedios y distribución por continente.

Opción 9 – Guardar y salir: se guardan los datos actualizados en el archivo CSV y se finaliza la ejecución del programa.

2.2 Decisiones sobre Estructuras y Algoritmos

- **Evitar librerías externas:** Se decidió no utilizar frameworks ni paquetes de terceros (como Pandas o OpenPyXL) para garantizar el cumplimiento estricto de las directrices pedagógicas de la cátedra y demostrar destreza en lógica algorítmica pura.
- **Manejo de Coincidencias Parciales:** En la búsqueda de países se implementó el operador de membresía in combinado con el método de normalización de cadenas .lower(), garantizando una experiencia de usuario elástica que ignora discrepancias por mayúsculas o minúsculas.

2.3 Evidencias de ejecución

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: █
```

Figura 1 – Menú principal del programa.

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 1
Nombre: Uruguay
Población: 3000000
Superficie: 176215
Continente: America
País agregado
```

Figura 2 – Agregado de un nuevo país (Uruguay).

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 2
Nombre: Polonia
Nueva población (vacío para no cambiar): 2500000
Nueva superficie (vacío para no cambiar): 87647
País actualizado
```

Figura 3 – Actualización de población y superficie de un país existente.

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 3
Texto a buscar: gentina
- Argentina | Población: 45376763 | Superficie: 2780400 | Continente: America
```

Figura 4 – Búsqueda de país por coincidencia parcial (ej: "gentina").

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 4
Continente: America
- Argentina | Población: 45376763 | Superficie: 2780400 | Continente: America
- Brasil | Población: 213993437 | Superficie: 8515767 | Continente: America
- Uruguay | Población: 3000000 | Superficie: 176215 | Continente: America
```

Figura 5 – Filtrado de países por continente (ej: América).

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 5
Mínima (opcional): 50000000
Máxima (opcional): 150000000
- Japon | Población: 125800000 | Superficie: 377975 | Continente: Asia
- Alemania | Población: 83149300 | Superficie: 357022 | Continente: Europa
- Corea del Sur | Población: 51740000 | Superficie: 100363 | Continente: Asia
```

Figura 6 – Filtrado por rango de población (ej: entre 50M y 150M).

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 6
Mínima (opcional): 300000
Máxima (opcional): 1000000
- Japon | Población: 125800000 | Superficie: 377975 | Continente: Asia
- Alemania | Población: 83149300 | Superficie: 357022 | Continente: Europa
```

Figura 7 – Filtrado por rango de superficie (ej: entre 300.000 y 1.000.000 km²).

```

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 7
Campo (nombre/poblacion/superficie): poblacion
Descendente? s/n: s
- Brasil | Población: 213993437 | Superficie: 8515767 | Continente: America
- Japon | Población: 125800000 | Superficie: 377975 | Continente: Asia
- Alemania | Población: 83149300 | Superficie: 357022 | Continente: Europa
- Corea del Sur | Población: 51740000 | Superficie: 100363 | Continente: Asia
- Argentina | Población: 45376763 | Superficie: 2780400 | Continente: America
- Uruguay | Población: 3000000 | Superficie: 176215 | Continente: America
- Polonia | Población: 2500000 | Superficie: 87647 | Continente: Europa

```

Figura 8 – Ordenamiento de países por población descendente.

```

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 8
Mayor población: Brasil
Menor población: Polonia
Promedio población: 75079928.57142857
Promedio superficie: 1770769.857142857
Por continente: {'America': 3, 'Asia': 2, 'Europa': 2}

```

Figura 9 – Estadísticas: país más/menos poblado, promedios y cantidad por continente.

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar por continente
5. Filtrar por población
6. Filtrar por superficie
7. Ordenar
8. Estadísticas
9. Salir
Elija una opción: 9
Datos guardados. ¡Hasta luego!
```

Figura 10 – Guardado de datos y salida del programa.

3. DIFICULTADES Y CONCLUSIONES

3.1 Dificultades Técnicas y Soluciones Aplicadas

A lo largo del ciclo de vida del desarrollo surgieron los siguientes obstáculos técnicos de carácter crítico:

- **Problema con caracteres especiales en el CSV:** Al leer nombres con tildes (como "Japón" o "América"), el programa fallaba o distorsionaba los caracteres en sistemas operativos con configuraciones de idioma nativas distintas.
Solución: Se forzó explícitamente el parámetro de codificación estandarizado `encoding='utf-8'` dentro del constructor de apertura de archivos nativo.
- **Persistencia en tiempo real vs. rendimiento:** Modificar el archivo físico en el almacenamiento secundario con cada inserción individual ralentizaba la experiencia de usuario.
Solución: Se decidió mantener las operaciones de alta y modificación exclusivamente sobre la estructura de la lista en la memoria RAM corporativa y ejecutar el volcado masivo al archivo CSV únicamente de manera controlada al seleccionar la opción de salida del sistema.

3.2 Conclusiones Grupales y Aprendizajes

El desarrollo cooperativo de este proyecto integrador permitió afianzar el flujo de trabajo ágil y estructurado exigido en entornos académicos superiores de la UTN. La separación estricta de responsabilidades lógicas entre los integrantes evidenció la importancia de acordar contratos de interfaces de datos claros (llaves idénticas en diccionarios y firmas de funciones estables) antes de la codificación propiamente dicha. Se concluye que la modularización no es solo una buena práctica estética, sino una necesidad arquitectónica para construir aplicaciones robustas, escalables y fácilmente legibles para terceros evaluadores.

4. BIBLIOGRAFÍA Y WEBGRAFÍA

- Python Software Foundation.
- Universidad Tecnológica Nacional.